

ME 593 Final Project Report

Sriram Kodey, Henry Chang

May 15 2024

This is the final project report for ME 593 Feed-forward Control. The report started with the selected paper review, *An Inversion-Based Learning Approach for Improving Impromptu Trajectory Tracking of Robots With Non-Minimum Phase Dynamics* [1], followed by the simulation results from the proposed method, and finally a summary of this project.

1 Paper Review

The paper presents a learning-based method to enhance impromptu trajectory tracking for robots with non-minimum phase dynamics, which have unstable inverse dynamics. Traditional inversion-based feedforward methods for non-minimum phase systems require knowledge of the system dynamics and cannot be directly applied to systems where model uncertainty is too large. This paper proposes a solution by using input-output data to learn a stable, approximate inverse of the system's baseline dynamics. This approach improves tracking accuracy, when compared to the baseline feedback stabilized system, without requiring a system model, which contrasts with existing methods that depend on known models and can lead to instability. The paper confirms the stability and effectiveness of the approach through theoretical analysis and experimental validation on two robotic platforms. Due to time limitations, we plan to implement the approach on a cart-pole system for this project.

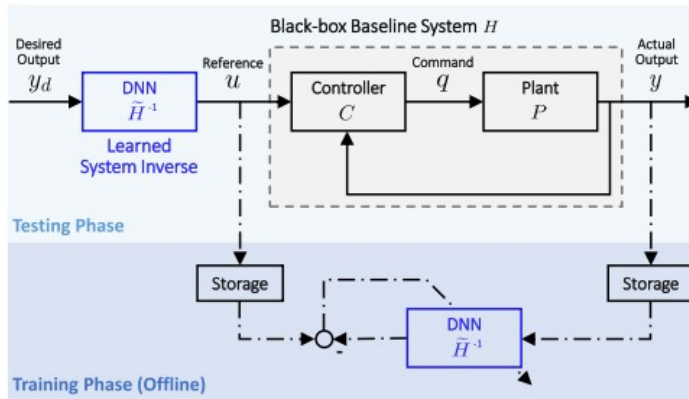


Figure 1: Illustration of controller architecture

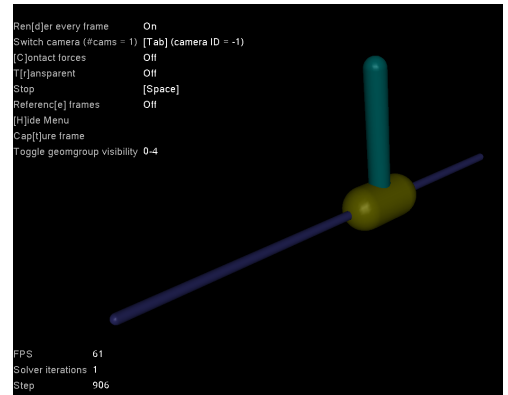


Figure 2: Cart pole system in Gymnasium

1.1 Overview

The premise of the paper is to utilize the input-output behavior of black box systems, stabilized using model-free controllers like PID, to learn the approximate inverse dynamics using a Feedforward Neural Network.

Considering that the nonlinear discrete model takes the control affine form given in eqn.1

$$\mathbf{x}(k+1) = f(\mathbf{x}(k)) + g(\mathbf{x}(k)) \cdot u(k), \quad y(k) = h(\mathbf{x}(k)), \quad (1)$$

For exact inverse learning, for a given desired trajectory y_d as presented in [2], the proper selection of inputs and outputs to the DNN model are as follows. $\mathcal{I} = \{\mathbf{x}(k), y_d(k+r)\}$ and $\mathcal{O} = \{u(k)\}$, where r , is the relative degree of the system. For LTI systems, this selection becomes : $\mathcal{I} = \{y_d(k-n+r : k+r)\}$ and $\mathcal{O} = \{u(k-n+r : k-1)\}$. But this exact inverse learning approach can't be extended to a non-minimum phase system as the DNN learned inverse from this input-output selection leads to instability when the feedforward controller is applied to the baseline system. To resolve this issue, a modified input-output selection strategy has been employed. The modified approach is as follows. We consider the linear baseline system for which, n is the order of the system and r is the relative degree, and given a desired trajectory $y_d(k)$, the exact inverse in the z domain is as follows:

$$H^{-1}(z) = \frac{U_{inv}(z)}{Y_d(z)} = \frac{D(z)}{N(z)} = \frac{\sum_{i=0}^n \beta_i z^i}{1 + \sum_{i=1}^{n-r} \alpha_i z^i} \quad (2)$$

The source of the instability in non-minimum phase systems is that at least one root of the Denominator $N(z)$, the numerator in the original system, is outside of the unit circle. The idea is to augment the DNN input selection strategy to ensure the network does not learn the unstable dynamics associated with these roots. This is the Zero Order Series approach extended to the neural network model. Taking the inverse z transform of the eq.(2), we get:

$$u(k) = F(\underbrace{y_d(k : k+n)}_{\text{from } D(z)}, \underbrace{u(k+1 : k+n-r)}_{\text{from } N(z)}) \quad (3)$$

Based on this the authors propose the following input-output selection strategy for training the DNN module, where the idea is to ignore the data that learns the dynamics of the unstable roots i.e the Denominator of the inverse system $N(z)$:

$$\mathcal{I} = \{y_d(k : k+n)\} \quad \text{and} \quad \mathcal{O} = \{u(k)\} \quad (4)$$

This input-output selection scheme has been derived for linear baseline systems but can be extended and applied to non-linear baseline systems with PID controllers since the neural network is trained directly on the input-output data generated on the nonlinear systems.

The authors also establish mathematical proof of stability for this approach, by leveraging properties of traditional neural network architectures and properties of activation functions like hyperbolic tangent ($Tanh$) used in them. They conclude that this approach guarantees the input-output stability of the DNN module conditioned on the stability of the baseline system with a model-free controller like PID. Thus, it is imperative to have a stable baseline system to implement the proposed approach effectively. In our simulations, we implemented a PID controller to generate the baseline system and verified its stability using simulations in the Gymnasium environment [3] using the MuJoCo physics plugin, as shown in Fig.2 and 3.

$$\ddot{\theta}_t = \frac{g \sin \theta_t + \cos \theta_t \left[\frac{-F_t - ml\dot{\theta}_t^2 \sin \theta_t + \mu_c \operatorname{sgn}(\dot{x}_t)}{m_c + m} \right] - \frac{\mu_p \dot{\theta}_t}{ml}}{l \left[\frac{4}{3} - \frac{m \cos^2 \theta_t}{m_c + m} \right]}$$

$$\ddot{x}_t = \frac{F_t + ml[\dot{\theta}_t^2 \sin \theta_t - \ddot{\theta}_t \cos \theta_t] - \mu_c \operatorname{sgn}(\dot{x}_t)}{m_c + m}$$

$g = -9.8 \text{ m/s}^2$, acceleration due to gravity,
 $m_c = 1.0 \text{ kg}$, mass of cart
 $m = 0.1 \text{ kg}$, mass of pole,
 $l = 0.5 \text{ m}$, half-pole length,
 $\mu_c = 0.0005$, coefficient of friction of cart on track,
 $\mu_p = 0.000002$, coefficient of friction of pole on cart,
 $F_t = \pm 10.0 \text{ newtons}$, force applied to cart's center of mass at time t .

Figure 3: Cart pole system detail in Gymnasium

1.2 Weakness

We discuss the strengths of this approach in the summary. However, the paper discusses several weaknesses of this approach, which are as follows:

1. Unlike typical learning approaches that are mostly robust to the inclusion of some uncorrelated data, this approach immediately breaks down with the inclusion of incorrect or excess data. Thus, the input-output selection and data collection processes are extremely important. They also note that for non-minimum phase systems *DNN trained with fewer inputs leads to better performance*. Thus, the inclusion of higher-order derivatives of the desired trajectory, $y_d(k+n+1:k+n+j)$, is detrimental to the performance of the controller.
2. Since the proposed approach is data-driven, its performance is heavily affected by the data it is trained on. Thus, the performance is better for the input trajectories that are closer to the trajectories in the training dataset. If the desired trajectory is quite dissimilar to the trajectories included in the training dataset, then the model performance is reduced significantly.

We also hypothesized that since the dynamics arising from the Denominator $N(z)$ have been ignored, there would be some shift in phase and magnitude (DC gain of the learned inverse system would be different). We observed the same and accounted for the gain difference by including a scaling factor.

We observed that the cost landscape is highly complex, making the training of the DNN model challenging and very sensitive to the training data and hyperparameters. This process demands significant time and effort for each type of desired trajectory. In contrast, exact inverse methods, even with model uncertainty, are more generalizable and perform well within the bounds of this uncertainty.

2 Training Data Collection and Simulation Results

The DNN module, as shown in Algorithm 1, is trained on 30 sinusoidal trajectories with different combinations of amplitudes $\{1, 2, 3, 4, 5, 6\}m$ and periods $\{0.15, 0.3, 0.45, 0.6, 0.75\}s$. The training dataset consists of pairs like Eq.4 randomly sampled from the 30 training trajectory. The model is validated using 30% of the training dataset, and further validation of the learning module is conducted by testing the entire system on untrained trajectories. Here, we reduced the amplitudes mentioned in the paper to accommodate the ± 1 meter limitation of the cart position in the Gymnasium environment.

To evaluate the proposed model performance, we applied a cart trajectory $y_d(k)$ to both baseline and learning-based systems with the sinusoidal period that is not in the training data sets, as shown in Fig.5. By implementing the learning-based inverse model, we observed substantial improvement in cart tracking

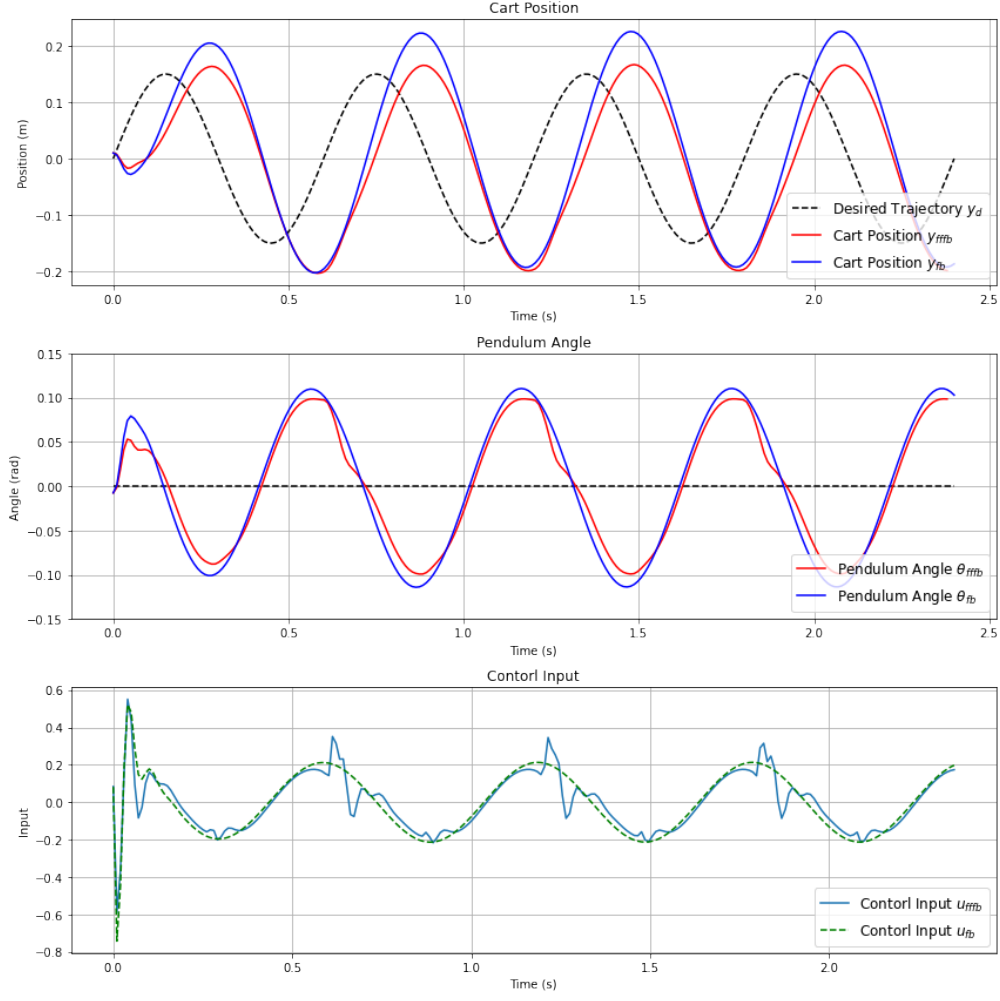


Figure 4: Simulation results for the closed-loop baseline system and the proposed learning-based system on a test trajectory $y_d(k) = 0.15 \sin(\frac{10\pi}{3}k)$. Top: cart position $y(k)$. Middle: pendulum angle $\theta(k)$. Bottom: system input $u(k)$.

performance, where the maximum overshoot is reduced by 78% (Baseline system: $0.0757m$, learning-based system: $0.0167m$). However, the RMS cart tracking error is reduced by only 6.6%. This is due to the persistent phase lag, as shown in Figure 3, and is expected since the DNN module learns an approximate inverse of the baseline system that ignores the dynamics of the Denominator $N(z)$. In comparison, the pre-actuation-based exact inverse method has no phase lag and performs much better in terms of tracking. But to maintain context, we reiterate that the exact inversion requires a system model, whereas the neural network-based approach is purely data-driven.

Since the training data ignores state information to avoid learning unstable dynamics, we also observe symmetry in u_{inv} , matching the symmetry in the desired trajectory. Since the inputs to the neural network module are $y_d(k : k+2)$, the learned approximate inverse will output the same u_{inv} regardless of the system’s state, provided that $y_d(k : k+2)$ matches.

3 Summary & Conclusion

We reviewed the paper *An Inversion-Based Learning Approach for Improving Impromptu Trajectory Tracking of Robots With Non-Minimum Phase Dynamics*. This paper proposes a method to improve trajectory tracking by using a Feedforward Neural Network (FNN) to learn the approximate inverse of the closed-loop system which does not require knowledge of the dynamics, unlike traditional methods.

We then implemented the proposed method by training the model with 30 sinusoidal trajectories and evaluating the performance using the cart-pole system. Our results indicate a significant reduction in maximum overshoot by 78% and a modest improvement in RMS cart tracking error by 6.6%. There is a consistent phase lag with this approach that arises due to the exclusion of dynamics that make the inverse system unstable and learning an approximate inverse. Even though the magnitude shift can be compensated for by adjusting the gain, the phase shift cannot be corrected.

In sum, we highlight the weaknesses of this approach in section 1.2, but this method also has its strengths. It has the edge over traditional methods in cases where the system cannot be modeled accurately or the discrepancy in the model is too high to implement traditional inverse methods. Since this is a data-based approach it might prove to be a useful tool in such applications. However, due to the sensitivity towards training data, it would fail if the order of the system is not accurately understood. Considering the robustness of model-based inversion methods, we can attempt data-driven physics-based discrepancy modeling and learn an intelligent model, which can then be used with exact inversion to achieve good tracking within the bounds of discrepancy.

References

- [1] Siqi Zhou, Mohamed K Helwa, and Angela P Schoellig. An inversion-based learning approach for improving impromptu trajectory tracking of robots with non-minimum phase dynamics. *IEEE Robotics and Automation Letters*, 3(3):1663–1670, 2018.
- [2] Siqi Zhou, Mohamed K Helwa, and Angela P Schoellig. Design of deep neural networks as add-on blocks for improving impromptu trajectory tracking. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, pages 5201–5207. IEEE, 2017.
- [3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.

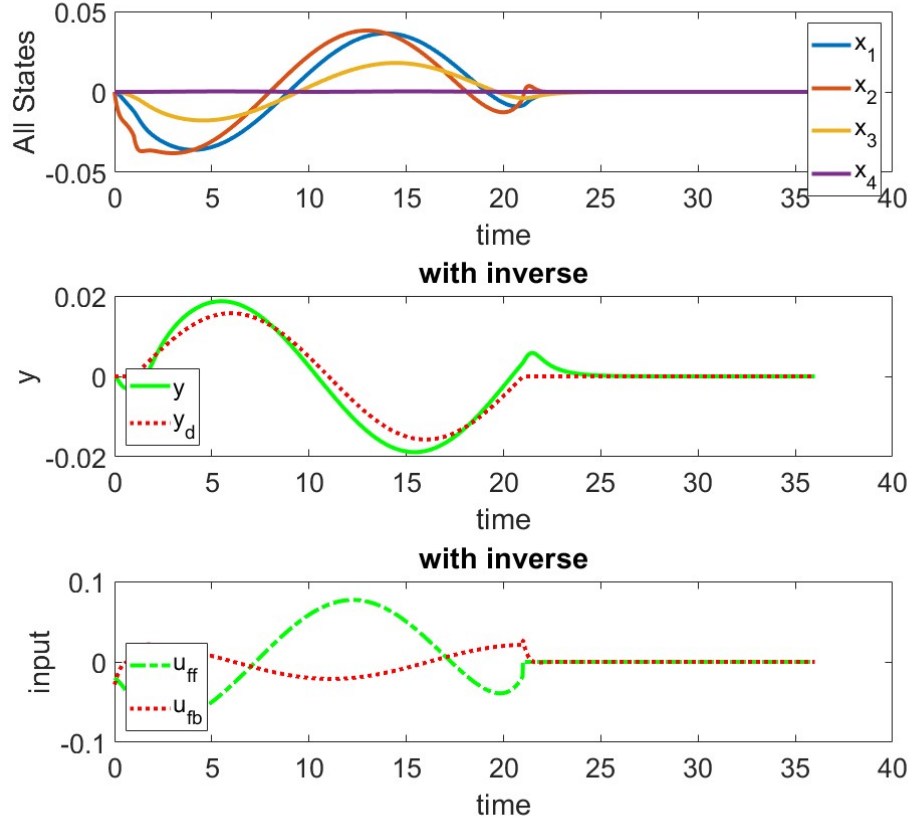


Figure 5: Simulation for exact inversion using pre-actuation. Top: the trajectory of all states, including internal states. Middle: Tracking performance when the desired trajectory is of a sine wave from. Bottom: Feedforward input and Feedback input

Algorithm 1 NeuralNet Class

```
1: class NeuralNet(nn.Module):
2:   def __init__(self, y_shape, u_shape):
3:     # Initialize parent class
4:     super(NeuralNet, self).__init__()
5:     # Flatten layer
6:     self.flatten = nn.Flatten()
7:     # Set input and output shapes
8:     self.input_shape = y_shape
9:     self.output_shape = u_shape
10:    # Define the neural network architecture
11:    self.NN = nn.Sequential(
12:      nn.Linear(self.input_shape, 5),
13:      nn.Tanh(),
14:      nn.Linear(5, 5),
15:      nn.Tanh(),
16:      nn.Linear(5, 5),
17:      nn.Tanh(),
18:      nn.Linear(5, self.output_shape)
19:    )
20:    # Define the loss function
21:    self.lossFn = nn.MSELoss()
22:    # Define the optimizer
23:    self.optimizer = optim.Adam(self.parameters(), lr=0.001)
24:  def forward(self, X):
25:    # Flatten the input
26:    x = self.flatten(X)
27:    # Pass through the neural network
28:    output = self.NN(x)
29:    return output
```
